

TOPIC 09

FUNCTION (PART 2)

Fundamentals of Computing
(FSPK0022)

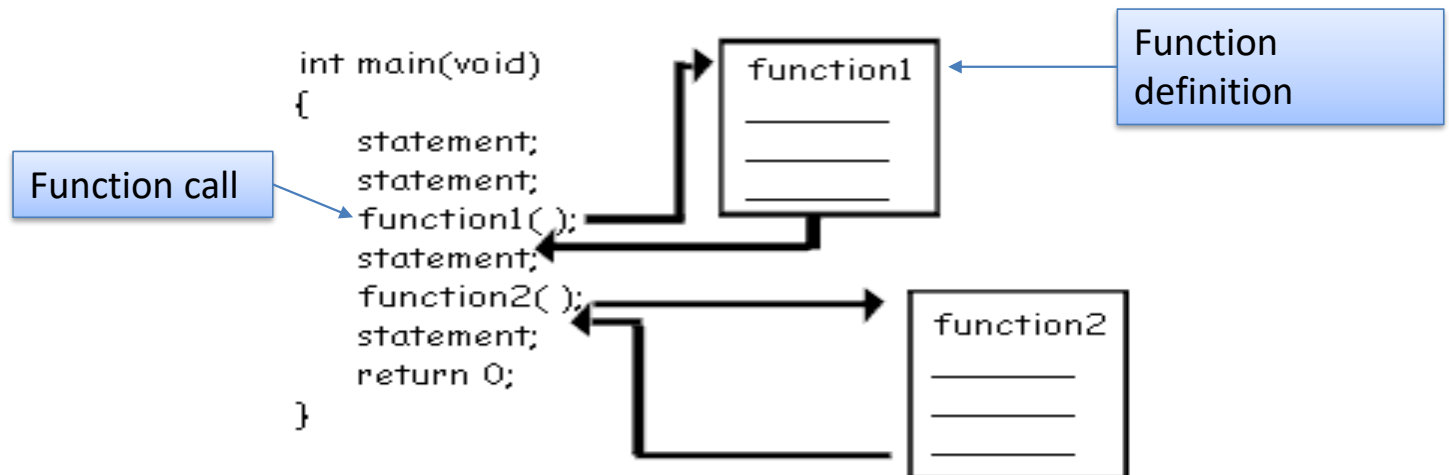
User-Defined Function

User-Defined Functions

- User-defined functions are created by you, the programmer.
- Commonly used to break a problem down into small **manageable** pieces.
- You are already familiar with the one function that every C++ program possesses: `int main()`
 - Ideally, your `main()` function should be very short and should consist primarily of function calls.

Defining and Calling Functions

- Every functions must have:
 - **Function call:** statement causes a function to execute
 - **Function definition:** statements that make up a function



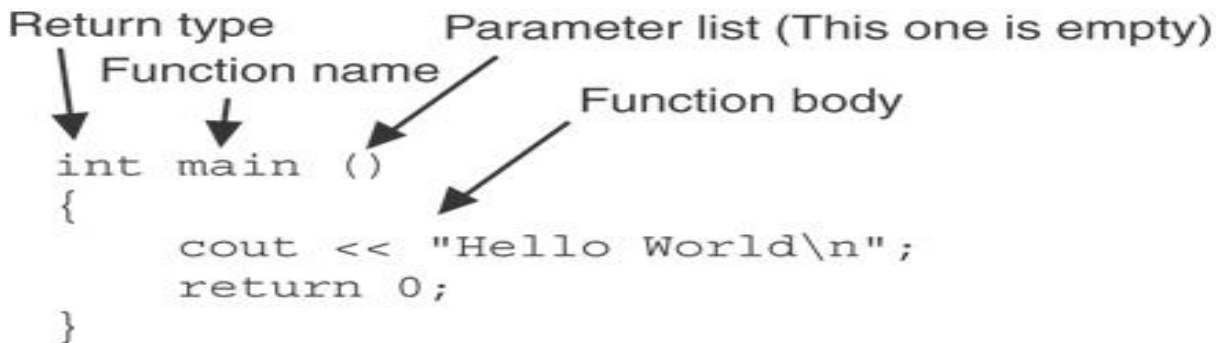
Function Definition

- Definition includes:
 - **return type:** data type of the value that function returns to the part of the program that calls it
 - **name:** name of the function. Function names follow same rules as variables
 - **parameter list:** variables containing values passed to the function
 - **body:** statements that perform the function's task, enclosed in { }

Function Definition (cont.)

- The general form of a function definition in C++ is as follows:

```
function-returntype function-name(arameter-list)  
{  
    local-definitions;  
    function-implementation;  
}
```



Return type Parameter list (This one is empty)

Function name Function body

```
int main ()  
{  
    cout << "Hello World\n";  
    return 0;  
}
```

Note: The line that reads `int main()` is the function header.

Function Return Type

- If a function returns a value, the type of the value must be indicated:

```
int main()
```

- If a function does not return a value, its return type is **void**. Example:

```
void printHeading()  
{  
    cout << "Monthly Sales\n";  
}
```

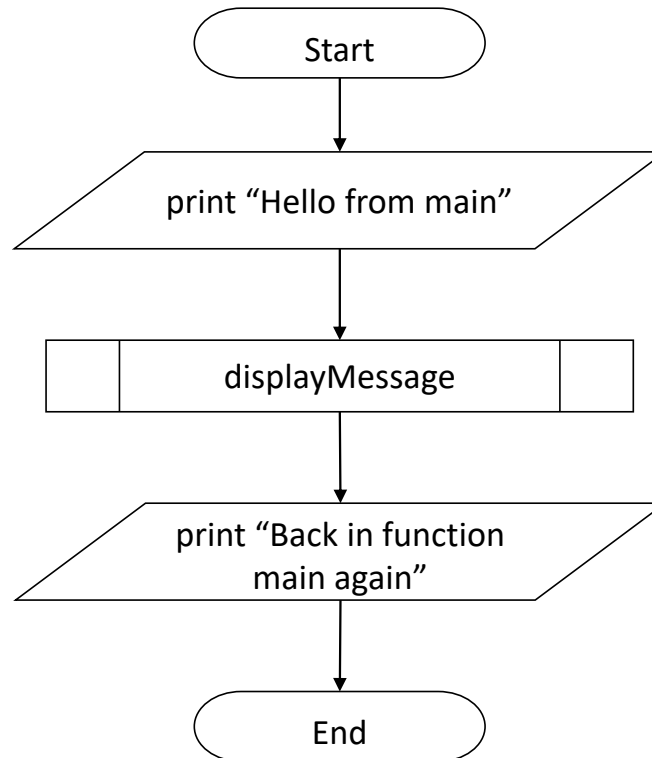
Calling a Function

- To call a function, use the function name followed by `()` and `;`

```
printHeading();
```

- When called, program executes the body of the called function
- After the function terminates, execution resumes in the calling function at point of call.

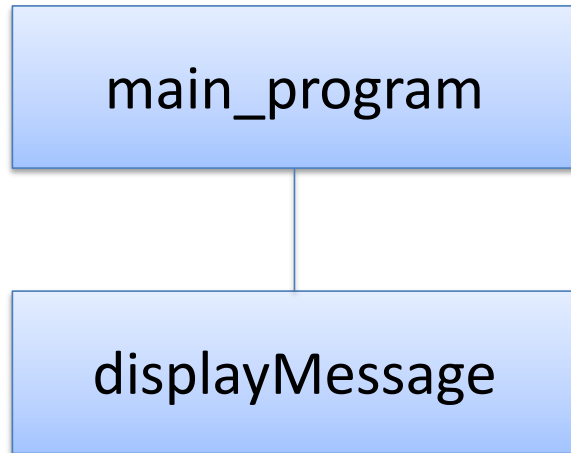
The Flowchart



The Pseudo Code

- Start
 - Print “Hello from main”
 - Call displayMessage
 - Print “Back in function main again”
 - End
- displayMessage:
 - Print “Hello from the function displayMessage”

The Structure Chart



Calling a Function - Example

Program 6-1

```
1 // This program has two functions: main and displayMessage
2 #include <iostream>
3 using namespace std;
4
5 //*****
6 // Definition of function displayMessage *
7 // This function displays a greeting. *
8 //*****
9
10 void displayMessage()
11 {
12     cout << "Hello from the function displayMessage.\n";
13 }
14
15 //*****
16 // Function main *
17 //*****
18
19 int main()
20 {
21     cout << "Hello from main.\n";
22     displayMessage();
23     cout << "Back in function main again.\n";
24     return 0;
25 }
```

Function
definition

Function call

Program Output

```
Hello from main.
Hello from the function displayMessage.
Back in function main again.
```

Flow of Control in Program 6-1



```
void displayMessage()  
{  
    cout << "Hello from the function displayMessage.\n";  
}
```

```
int main()  
{  
    cout << "Hello from main.\n"  
    displayMessage();  
    cout << "Back in function main again.\n";  
    return;  
}
```

User-Defined Functions:

Example 2

```
01  #include <iostream>
02  #include <cmath>
03  using namespace std;
04
05  float distance(float x, float y)
06  {    float dist;
07      dist = sqrt(x * x + y * y);
08      return dist;
09  }
10
11  void main()
12  {
13      float x,y,dist;
14      cout << "Testing function distance(x,y)" << endl;
15      cout << "Enter values for x and y: ";
16      cin >> x >> y;
17      dist = distance(x,y);
18      cout << "Distance of (" << x << ',' << y << ") from origin is
19      " << dist << endl << "Tested" << endl;
19  }
```

In-Class Exercise

- Which of the following function headers are valid? If they are invalid, explain why.
 - `one (int a, int b)`
 - `int thisone(char x)`
 - `char another (int a, b)`
 - `double yetanother`

Function Prototypes

- Ways to **notify the compiler** about a function before a call to the function:
 - Place function definition before calling function's definition
 - Use a function prototype (function declaration) – like the function definition without the body
 - Header: `void printHeading()`
 - Prototype: `void printHeading();`

User-Defined Functions: Function Prototypes

```
01  #include <iostream>
```

```
02
```

```
03  void first();
```

```
04  void second();
```

Function prototypes

```
05
```

```
06  void main()
```

```
07  {
```

```
08      cout<< "Starting in main function \n";
```

```
09      first();
```

```
10      second();
```

```
11      cout<<"Control back to main\n";
```

```
12  }
```

```
13
```

```
14  void first()
```

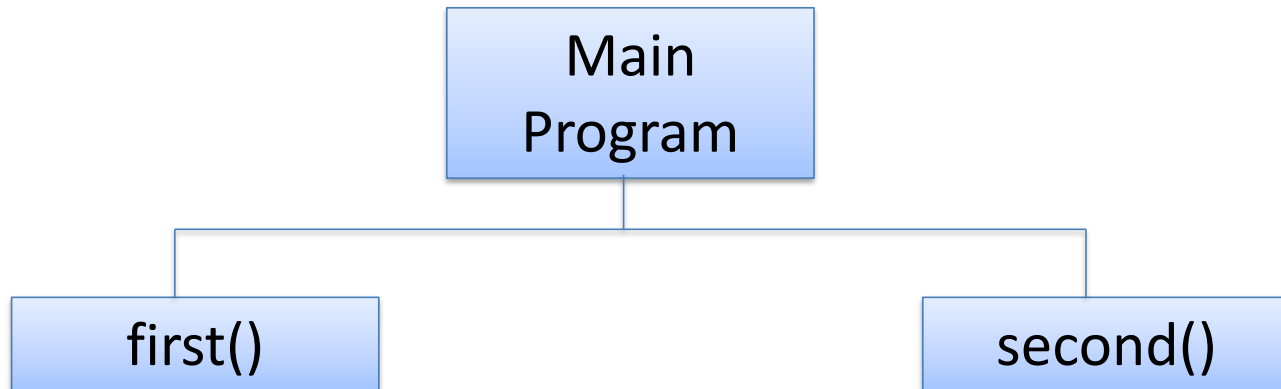
```
15  {  cout<<"Inside the first function\n";}
```

```
16
```

```
17  void second()
```

```
18  {  cout<<"Inside the second function\n";}
```

Structured Chart



Prototype Notes

- Place prototypes near **top** of program
- Program must include either prototype **or** full function definition before any call to the function – compiler error otherwise
- When using prototypes, can place function definitions in any order in source file

User-Defined Functions:

Functions with No Parameters

```
01 #include <iostream>
02 void printhi();
03
04 void main(){
05     cout << "Testing function printhi()" << endl;
06     printhi();
07     cout << "Tested" << endl;
08 } // End of main
09
10 // Function Definitions
11 void printhi()
12 {
13     cout << "Hi \n";
14 }
```

Sending Data into a Function

- Can **pass values** into a function at time of call:

```
c = pow(a, b) ;
```

- Values passed to function are arguments
- Variables in a function that **hold the values passed** as arguments are parameters

A Function with a Parameter Variable

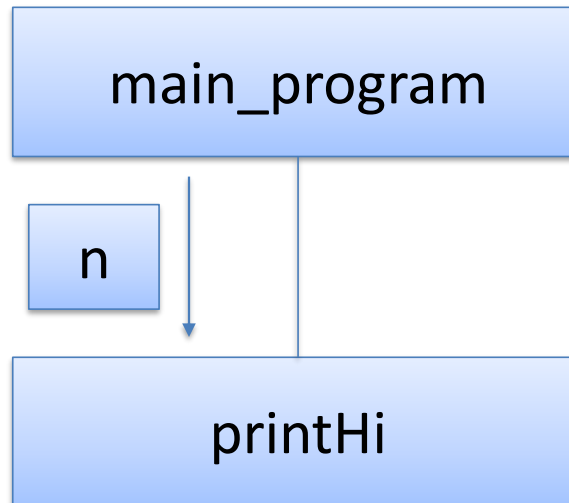
```
01 void displayValue(int num)
02 {
03     cout << "The value is " << num << endl;
04 }
```

The integer variable `num` is a **parameter**.
It accepts any integer value passed to the function.

Example: Functions with Parameter and No Return Values (1)

```
01  #include <iostream>
02  using namespace std;
03  void printhi(int);
04
05  void main(){
06      int n;
07      cout <<"Enter a value for n: ";
08      cin >> n;
09      printhi(n);
10      cout << "Tested \n";}
11
12  void printhi(int n)
13  {
14      int i;
14      for (i = 0; i < n; i++)
15      cout << "Hi \n";
16  }
```

The Structure Chart



Example: Functions with Parameter and No Return Values (2)

```
01 #include <iostream>
02 using namespace std;
03
04 void displayValue(int);
05
06 void main()
07 {
08     cout<<"Passing number 5 to displayValue\n";
09     displayValue(5);
10     cout<<"Back in main\n";
11 }
12
13 void displayValue(int n)
14 {
15     cout<<"The value is " << n << endl;
16 }
```

Other Parameter Terminology

- A parameter can also be called a formal parameter or a formal argument
- An argument can also be called an actual parameter or an actual argument

Parameters, Prototypes, and Function Headers

- For each function argument,
 - the **prototype** must include the **data type** of each parameter inside its parentheses
 - the **header** must include a **declaration** for each parameter in its ()

```
void evenOrOdd(int);    //prototype  
void evenOrOdd(int num) //header  
evenOrOdd(val);        //call
```

Function Call Notes

- Value of argument is **copied** into parameter when the function is called
- A parameter's scope is the function which uses it
- Function can have **multiple** parameters
- There **must** be a **data type** listed in the prototype () and an **argument declaration** in the function header () for each parameter
- Arguments will be promoted/demoted as necessary to **match** parameters

In-Class Exercise

- What is the output of this program?

```
01  #include <iostream>
02
03  // Function prototype
04  void showDouble(int);
05
06  int main(){
07      int num;
08      for (num = 0; num < 10; num++)
09          showDouble(num);
10      system("pause");
11      return 0;
12  }
13
14  //Definition of function
15  void showDouble(int value) {
16      cout<<value <<"\t";
17      cout << (value * 2)<< endl;
18  }
```

Passing Multiple Arguments

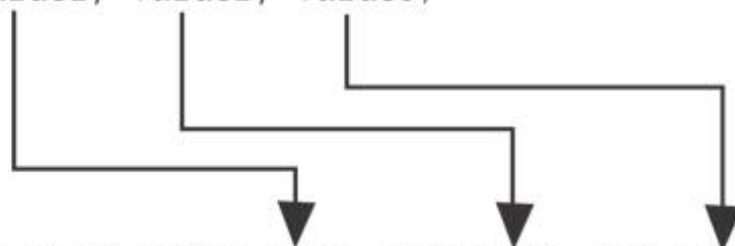
- When calling a function and passing multiple arguments:
 - the number of arguments in the call must match the prototype and definition
 - the first argument will be used to initialize the first parameter, the second argument to initialize the second parameter, etc.

Example: Passing Multiple Arguments

```
01  #include <iostream>
02  using namespace std;
03  void showSum(int, int, int);
04
05  int main()
06  {
07      int value1, value2, value3;
08
09      cout<<"Enter 3 integers: ";
10      cin>> value1 >> value2 >> value3;
11      showSum(value1, value2, value3);
12
13      return 0;
14  }
15
16  void showSum(int a, int b, int c)
17  {
18      cout<<"The sum: "<<a+b+c;
19  }
```

Passing Multiple Arguments

Function Call → `showSum(value1, value2, value3)`



```
void showSum(int num1, int num2, int num3)
{
    cout << (num1 + num2 + num3) << endl;
}
```

The function call in line 18 passes value1, value2, and value3 as arguments to the function.

In-Class Exercise

- What is the output of this program?

```
01  #include <iostream>
02
03  // Function prototype
04  void func1(double, int);
05
06  int main(){
07      int x = 0; double y = 1.5;
08      cout << x << " " <<y<< endl;
09      func1 (y, x);
10      cout << x << " " <<y<< endl;
11      system ("pause");
12      return 0;
13  }
14
15  void func1(double a, int b){
16      cout << a << " " <<b<< endl;
17      a=0.0; b=10;
18      cout << a << " " <<b<< endl;
19  }
```

Passing Data by Value

- Pass by value: when an argument is passed to a function, its value is **copied** into the parameter.
- Changes to the parameter in the function do not affect the value of the argument

Passing Data by Value (cont.)

```
01  #include <iostream>
02  using namespace std;
03
04  void f( int n ) {
05      cout << "Inside f( int ), the value of the parameter is "
06          << n << endl;
07      n += 37;
08      cout << "Inside f( int ), the modified parameter is now "
09          << n << endl;}
10
11  int main() {
12      int m = 612;
13
14      cout << "The integer m = " << m << endl;
15      cout << "Calling f( m )..." << endl;
16      f( m );
17      cout << "The integer m = " << m << endl;
18      return 0;
19  }
```

Passing Data by Value (cont.)

Inside **main()**:

m 612

Call **f(m)**;

memory allocated for **n**

copy the value **612** to this location

Inside **f(int n)**:

m 612

n 612

f(int) modifies **n**:

m 612

n 649

Deallocate memory for **n**

Return to **main()** ;

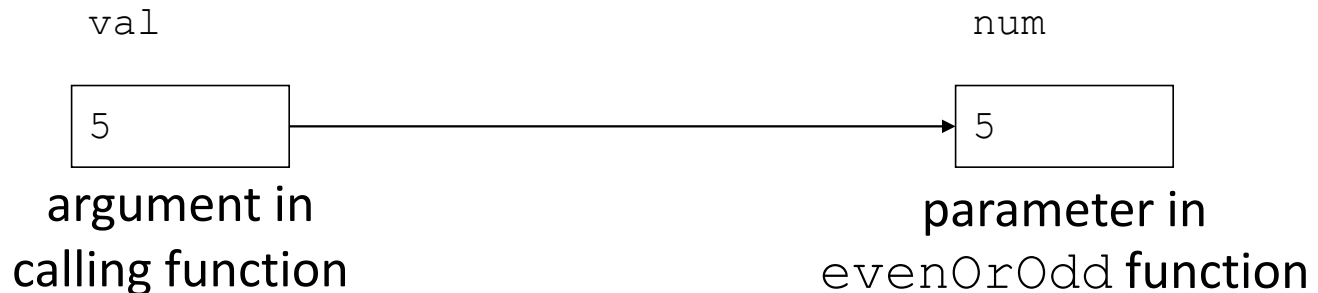
Back in **main()**:

the variable **m** is unchanged:

m 612

Passing Information to Parameters by Value

- Example: `int val=5;`
`evenOrOdd(val);`



- `evenOrOdd` can change variable `num`, but it will have no effect on variable `val`

Using Functions in Menu-Driven Programs

- Functions can be used
 - to implement user choices from menu
 - to implement general-purpose tasks:
 - Higher-level functions can call general-purpose functions, minimizing the total number of functions and speeding program development time

The `return` Statement

- Used to end execution of a function
- Can be placed anywhere in a function
 - Statements that follow the `return` statement will not be executed
- Can be used to prevent abnormal termination of program
- In a `void` function without a `return` statement, the function ends at its last }

Returning a Value from a Function

- A function can return a value back to the statement that called the function.
- You've already seen the **pow** function, which returns a value:

```
double x;  
x = pow(2.0, 10.0);
```


Returning a Value from a Function

- In a value-returning function, the **return** statement can be used to return a value from function to the point of call. Example:

```
int sum(int num1, int num2)
{
    double result;
    result = num1 + num2;
    return result;
}
```

A Value-Returning Function

Return Type



```
int sum(int num1, int num2)
{
    double result;
    result = num1 + num2;
    return result;
}
```

Value Being Returned



A Value-Returning Function

```
int sum(int num1, int num2)
{
    return num1 + num2;
}
```

Functions can return the values of expressions,
such as `num1 + num2`

The return Statement - Example

Program 6-11

```
1  // This program uses a function to perform division. If division
2  // by zero is detected, the function returns.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype.
7  void divide(double, double);
8
9  int main()
10 {
11     double num1, num2;
12
13     cout << "Enter two numbers and I will divide the first\n";
14     cout << "number by the second number: ";
15     cin >> num1 >> num2;
16     divide(num1, num2);
17     return 0;
18 }
```

The return Statement - Example

Program 6-11(Continued)

```
20  //*****
21  // Definition of function divide.
22  // Uses two parameters: arg1 and arg2. The function divides arg1*
23  // by arg2 and shows the result. If arg2 is zero, however, the *
24  // function returns.
25  //*****
26
27  void divide(double arg1, double arg2)
28  {
29      if (arg2 == 0.0)
30      {
31          cout << "Sorry, I cannot divide by zero.\n";
32          return;
33      }
34      cout << "The quotient is " << (arg1 / arg2) << endl;
35  }
```

Return to
called
function



Program Output with Example Input Shown in Bold

Enter two numbers and I will divide the first
number by the second number: **12 0 [Enter]**
Sorry, I cannot divide by zero.

Returning a Value From a Function

Program 6-12

```
1  // This program uses a function that returns a value.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototype
6  int sum(int, int);
7
8  int main()
9  {
10     int value1 = 20,    // The first value
11         value2 = 40,    // The second value
12         total;          // To hold the total
13
14     // Call the sum function, passing the contents of
15     // value1 and value2 as arguments. Assign the return
16     // value to the total variable.
17     total = sum(value1, value2);
18
19     // Display the sum of the values.
20     cout << "The sum of " << value1 << " and "
21         << value2 << " is " << total << endl;
22     return 0;
23 }
```

Returning a Value From a Function

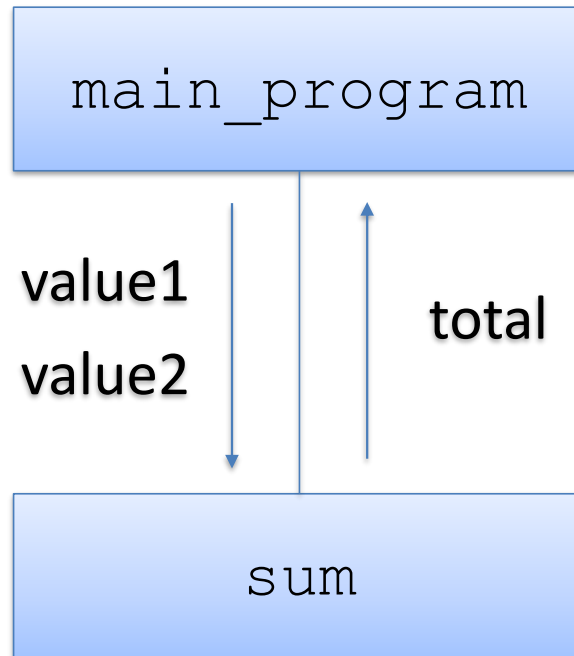
Program 6-12 (Continued)

```
24
25  //*****
26  // Definition of function sum. This function returns  *
27  // the sum of its two parameters.                      *
28  //*****
29
30  int sum(int num1, int num2)
31  {
32      return num1 + num2;
33  }
```

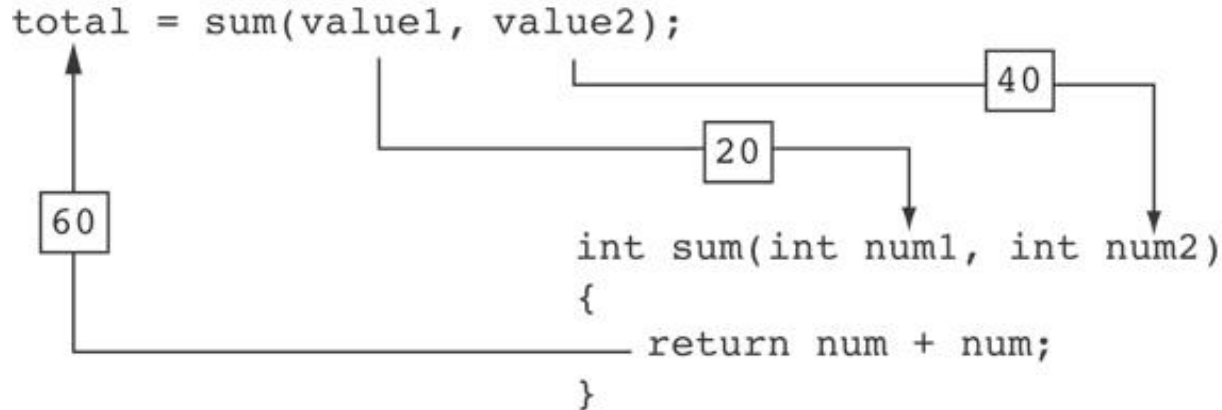
Program Output

The sum of 20 and 40 is 60

The Structure Chart



Returning a Value From a Function



The statement in line 17 calls the `sum` function, passing `value1` and `value2` as arguments.

The **return** value is assigned to the `total` variable.

Returning a Value From a Function

- The prototype and the definition must indicate the data type of return value (not `void`)
- Calling function should use return value:
 - assign it to a variable
 - send it to `cout`
 - use it in an expression

Returning a Boolean Value

- Function can return `true` or `false`
- Declare return type in function prototype and heading as `bool`
- Function body must contain `return` statement(s) that return `true` or `false`
- Calling function can use return value in a relational expression

Returning a Boolean Value

Program 6-14

```
1  // This program uses a function that returns true or false.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototype
6  bool isEven(int);
7
8  int main()
9  {
10     int val;
11
12     // Get a number from the user.
13     cout << "Enter an integer and I will tell you ";
14     cout << "if it is even or odd: ";
15     cin >> val;
16
```

Returning a Boolean Value

Program 6-14 (continued)

```
17     // Indicate whether it is even or odd.
18     if (isEven(val))
19         cout << val << " is even.\n";
20     else
21         cout << val << " is odd.\n";
22     return 0;
23 }
24
25 /*******
26 // Definition of function isEven. This function accepts an      *
27 // integer argument and tests it to be even or odd. The function *
28 // returns true if the argument is even or false if the argument *
29 // is odd. The return value is an bool.                          *
30 /*******
31
32 bool isEven(int number)
33 {
34     bool status;
35
36     if (number % 2)
37         status = false;    // number is odd if there's a remainder.
38     else
39         status = true;     // Otherwise, the number is even.
40     return status;
41 }
```

Program Output with Example Input Shown in Bold

Enter an integer and I will tell you if it is even or odd: **5 [Enter]**
5 is odd.

In-Class Exercise

```
#include <iostream>
using namespace std;
void try1(int p);
int try3(int r);

int main()
{ int a=2;
  cout << a <<endl;
  try1(a);
  cout << a <<endl;
  int b=3;
  cout << b <<endl;
  int c=4;
  try3(c);
  cout << c <<endl;
  c=try3(c);
  cout << c <<endl;
  cout << try3(5) <<endl;
  return 0;}
```

```
void try1(int p)
{
    p++;
    cout << p <<endl;
}

int try3(int r)
{
    return r*r;
}
```

In-Class Exercise

- Write a function prototype and header for a function named `distance`. The function should return a `double` and have a two `double` parameters: `rate` and `time`.
- Write a function prototype and header for a function named `days`. The function should return an integer and have three integer parameters: `years`, `months` and `weeks`.
- Examine the following function header, then write an example call to the function.
 - `void showValue(int quantity)`

In-Class Exercise

- The following statement calls a function named `half`. The `half` function returns a value that is half that of the argument. Write the function.

```
result = half(number);
```

- A program contains the following function:

```
int cube (int num)
{
    return num*num*num;
}
```

Write a statement that passes the value 4 to this function and assigns its return value to the variable `result`.

In-Class Exercise

- Write a C++ program to calculate a rectangle's area. The program consists of the following functions:
 - **getLength** – This function should ask the user to enter the rectangle's length, and then returns that value as a double.
 - **getWidth** – This function should ask the user to enter the rectangle's width, and then returns that value as a double.
 - **getArea** – This function should accept the rectangle's length and width as arguments and return the rectangle's area.
 - **displayData** – This function should accept the rectangle's length, width and area as arguments, and display them in an appropriate message on the screen.
 - **main** – This function consists of calls to the above functions.

Local and Global Variables

- Variables defined inside a function are *local* to that function. They are hidden from the statements in other functions, which normally cannot access them.
- Because the variables defined in a function are hidden, other functions may have separate, distinct variables with the same name.

Local and Global Variables - Example

Program 6-15

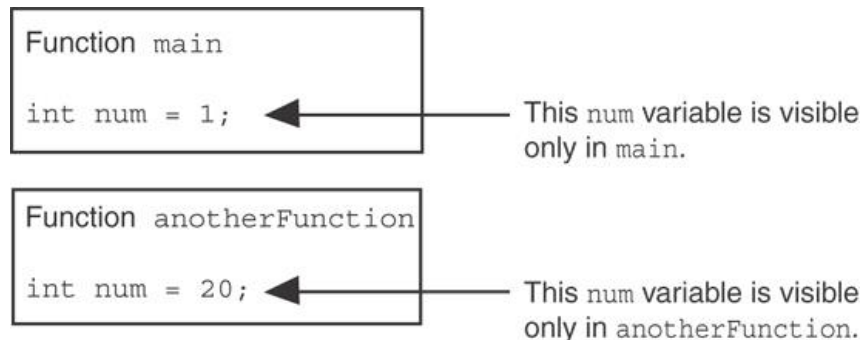
```
1 // This program shows that variables defined in a function
2 // are hidden from other functions.
3 #include <iostream>
4 using namespace std;
5
6 void anotherFunction(); // Function prototype
7
8 int main()
9 {
10     int num = 1;    // Local variable
11
12     cout << "In main, num is " << num << endl;
13     anotherFunction();
14     cout << "Back in main, num is " << num << endl;
15     return 0;
16 }
17
18 //*****
19 // Definition of anotherFunction                *
20 // It has a local variable, num, whose initial value *
21 // is displayed.                                *
22 //*****
23
24 void anotherFunction()
25 {
26     int num = 20;    // Local variable
27
28     cout << "In anotherFunction, num is " << num << endl;
29 }
```

Program Output

```
In main, num is 1
In anotherFunction, num is 20
Back in main, num is 1
```

Local and Global Variables - Example

- When the program is executing in `main`, the `num` variable defined in `main` is visible.
- When `anotherFunction` is called, however, only variables defined inside it are visible, so the `num` variable in `main` is hidden.



Local Variable Lifetime

- A function's local variables exist only while the **function is executing**. This is known as the *lifetime* of a local variable.
- When the function begins, its local variables and its parameter variables are **created** in memory, and when the function ends, the local variables and parameter variables are **destroyed**.
- This means that any value stored in a local variable is lost between calls to the function in which the variable is declared.

Global Variables and Global Constants

- A **global** variable is any variable defined **outside** all the functions in a program.
- The scope of a global variable is the portion of the program from the variable definition to the end.
- This means that a global variable can be accessed by **all** functions that are defined after the global variable is defined.
- You should **avoid** using global variables because they make programs difficult to debug.
- Any global that you create should be **global constants**.

Global Constants – Example

Program 6-18

```
1 // This program calculates gross pay.
2 #include <iostream>
3 #include <iomanip>
4 using namespace std;
5
6 // Global constants
7 const double PAY_RATE = 22.55;    // Hourly pay rate
8 const double BASE_HOURS = 40.0;   // Max non-overtime hours
9 const double OT_MULTIPLIER = 1.5; // Overtime multiplier
10
11 // Function prototypes
12 double getBasePay(double);
13 double getOvertimePay(double);
14
15 int main()
16 {
17     double hours,           // Hours worked
18           basePay,          // Base pay
19           overtime = 0.0,   // Overtime pay
20           totalPay;         // Total pay
```

Global constants defined for values that do not change throughout the program's execution.

Global Constants – Example

The constants are then used for those values throughout the program.

```
29      // Get overtime pay, if any.
30      if (hours > BASE_HOURS)
31          overtime = getOvertimePay(hours);
```

```
56      // Determine base pay.
57      if (hoursWorked > BASE_HOURS)
58          basePay = BASE_HOURS * PAY_RATE;
59      else
60          basePay = hoursWorked * PAY_RATE;
```

```
75      // Determine overtime pay.
76      if (hoursWorked > BASE_HOURS)
77      {
78          overtimePay = (hoursWorked - BASE_HOURS) *
79                          PAY_RATE * OT_MULTIPLIER;
```


Initializing Local and Global Variables

- Local variables are not automatically initialized. They must be initialized by programmer.
- Global variables (not constants) are automatically initialized to 0 (numeric) or `NULL` (character) when the variable is defined.

In-Class Exercise

- Identify global variables & local variables in the following program. What is the output?

```
#include <iostream>
using namespace std;
int j = 8;

int main()
{
    int i = 0;
    cout<<"i: "<<i<<endl;
    cout<<"j: "<<j<<endl;
    return 0;
}
```